

67-101
תש"ס/כ"ב

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67108 – מבוא למדעי המחשב לתלמידי תלפיות

67112 – מבוא למדעי המחשב לתלמידי תלפיות מצומצם

תאריך: 21.5.2010

מועד ב' - סמסטר א' - תש"ע – 2009-2010

משך הבחינה: שלוש שעות

פרופ' דפנה ויינשל, פרופ' יובל רבני, אלעד מזומן

הנחיות:

1. יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
2. יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
3. יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
4. הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
5. יש למחוק טיוטות באופן ברור.
6. יש להשאיר שוליים.
7. אסור להשתמש בעט אדום.
8. ניתן לכתוב בעיפרון אך לא ניתן יהיה לערער על בחינות שנכתבו בעיפרון.
9. הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
10. תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
11. פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
12. אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
13. בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.

67.101
10/8'PA

שאלה 1 (33 נקודות)

פולימורפיזם

קראו בעיון את הקוד הבא וענו על השאלות שאחריו:

```
abstract class Vehicle {
    void sketch() {
        System.out.println("Vehicle.sketch()");
    }

    abstract void move();
}

class Truck extends Vehicle {
    void sketch() {
        System.out.println("Truck.sketch()");
    }

    void move() {
        System.out.println("Truck.move()");
    }
}

class Car extends Vehicle {
    void move() {
        System.out.println("Car.move()");
    }
}

class Sedan extends Car {
    void sketch() {
        System.out.println("Sedan.sketch()");
    }

    void move() {
        System.out.println("Sedan.move()");
    }
}

class Bus extends Vehicle {
    void sketch() {
        System.out.println("Bus.sketch()");
    }
}
```

המשך השאלה בעמוד הבא

67. 101
12/8/21

```
// A "factory" that randomly creates vehicles: assume that
// rand.nextInt(6) returns a random integer between 0 and 5
import java.util.Random;

class RandomVehicleGenerator {
    private Random rand = new Random();

    public Vehicle next() {
        switch (rand.nextInt(6)) {
            default: return new Truck();
            case 0: return new Vehicle();
            case 1: return new Truck();
            case 2: return new Car();
            case 3: return new Sedan();
            case 4: return new Bus();
        }
    }
}

public class VehicleSketch {
    private RandomVehicleGenerator gen = new RandomVehicleGenerator();

    public static void main(String[] args) {
        Vehicle[] s = new Vehicle[7];
        for (int i = 0; i < s.length; i++)
            s[i] = gen.next();
        for (int i = 0; i < s.length; i++)
            s[i].sketch();
    }
}
```

(א) (9 נק') זהו שלושה מקומות שגויים (בעיות קומפילציה) והציעו תיקונים (מינימליים) שיבטיחו קומפילציה וריצה תקינה. אסור לשנות את חתימות המתודות או את הגדרת המחלקות.

(א) (10 נק') לאחר התיקון, מה יודפס אם סדרת הספרות שתיוצר ע"י `rand.nextInt(6)` היא הבאה: 0,5,2,1,3,2,4

(ב) (9 נק') כחלק ממהלך להרחבת התוכנה, הנכם מתבקשים לשנות את הקוד כך שהמחלקה `VehicleSketch` תוכל לצייר גם אובייקטים מטיפוסים נוספים. תארו איך ניתן לשנות את הקוד שלמעלה כדי לתמוך באפשרות כזו, מבלי לשנות את עץ הירושה של המחלקה `Vehicle` וצאצאיה. משיקוליי תחזוקה עליכם להקפיד על שינוי מינימלי – מומלץ לשנות כמה שפחות קבצים, וכמה שפחות קוד בכל קובץ. הסבירו מה יידרש ממחלקות שיוכלו להשתתף בתכנית ציור המכוניות `VehicleSketch`.

(ג) (5 נק') עליכם להגדיר מחלקה חדשה בשם `C4x4` אשר לה ול-`Car` הורה משותף – מחלקה בשם `PrivateVehicle`. אין לאפשר יצירת גופים מסוג `PrivateVehicle`. תארו את עץ הירושה החדש של `Vehicle`, וכיתבו את הקוד עבור השיטות ששוננו והמחלקות החדשות שנוצרו.

67.101
ס/ס' 21

שאלה 2 (33 נקודות)

רקורסיה: גן החיות של קיסר סין

גן החיות של קיסר סין עבר לאחרונה שיפוץ מאסיבי. בגן הוקמו שלושה כלובי ענק שונים עבור החיות. מנהל גן החיות רוצה לחלק את החיות השונות בין שלושת הכלובים. הבעיה היא שלא תמיד ניתן לשכן שני מינים שונים של חיות יחד. לחלק מהחיות נדרשים תנאים מיוחדים שונים אחרים אינם יכולים לסבול ובנוסף, חיות מסוימות עלולות לטרוף חיות אחרות. עזרו למנהל גן החיות לשבץ את החיות לכלובים השונים.

נתונה המחלקה Animal אשר לה שתי השיטות הבאות

(מחזיר את שם החיה)

```
public String getName()
```

(בודק אם זוג חיות יכולות לחיות יחד)

```
public boolean canLiveWith(Animal other)
```

כך שקריאה למתודה

```
animal1.canLiveWith(animal2) או לחלופין animal2.canLiveWith(animal1)
```

מחזירה ערך אמת אם ורק אם החיות animal1, animal2 יכולות לחיות יחד באותו כלוב (שתי הקריאות הנ"ל שקולות ומחזירות את אותו ערך). המחלקה Animal כבר קיימת ואין צורך לממש את המתודות שלה.

משימתכם היא לכתוב מתודה סטטית אשר תמצא עבור מנהל גן החיות השמה של החיות לכלובים השונים, כך שכל החיות באותו כלוב יכולות לחיות ביחד. בשאלה זו מותר להקצות מערכים בגודל לינארי באורך הקלט (מס' החיות) בשאלה יתקבל גם פתרון הרץ בזמן אקספוננציאלי. חתימת המתודה שעליכם לכתוב היא:

```
public static void findCageAssignment(Animal[] animals)
```

המערך animals מחזיק את האובייקטים המייצגים את החיות בגן. אם לא קיימת השמה חוקית, המתודה תדפיס הודעה שאומרת שאין השמה כזו. אם קיימת קיימת השמה חוקית, היא תודפס למסך כך שכל חיה ומספר הכלוב אליו היא משובצת יופיעו בשורה נפרדת:

<animal name> <cage number>

סדר ההדפסה של השורות השונות אינו חשוב, ויש להקפיד להדפיס רק השמה אחת לכלובים. בהשמה חוקית אין חובה לאכלס את כל הכלובים, אבל חייבים לשבץ את כל החיות (מספרי הכלובים הם 0,1,2).

לדוגמא: אם במערך animals ניתנו החיות הבאות: Tiger, Zebra, Frog, Butterfly, Dolphin (נמר, זברה, צפרדע, פרפר, ודולפין) ואם היחס בין החיות (כפי שמוגדר ע"י השיטה canLiveWith) הוא: הדולפין לא יכול לחיות עם אף חיה אחרת (הוא צריך סביבה מימית), הנמר עלול לטרוף את הזברה אבל לא יגע בצפרדע או בפרפר, הזברה חיה בשלום עם הפרפר והצפרדע. הצפרדע עלולה לאכול את הפרפר. השמה אפשרית של החיות למתחמים היא:

Tiger 1, Zebra 0, Frog 0, Butterfly 1, Dolphin 2

כלומר, השמה זו ממקמת את הנמר והפרפר יחד בכלוב 1, את הזברה והצפרדע בכלוב 0, ואת הדולפין לבדו בכלוב 2.

(א) (28 נק') כתבו את השיטה findCageAssignment (ניתן להגדיר משתני מחלקה ושיטות עזר נוספות). על הפיתרון לעבוד באופן רקורסיבי, פיתרונות אחרים לא יתקבלו.

(ב) (5 נק') האם הפיתרון שעצעתם בחלק הקודם הוא פולינומיאלי בגודל הקלט או לא? תנו דוגמה לקלט בן 6 חיות עבורו זמן הריצה של השיטה שמישתם קטן משמעותית מהמקרה הכללי של קלט עם 6 חיות.

67. 101
2/2 PA

שאלה 3 (33 נקודות)

לצורך שאלה זו הניחו כי נתונה לכם מחלקה בשם CharStack, המממשת מחסנית עבור טיפוס נתונים מסוג char (אין צורך לממש מחלקה זו). למחלקה השיטות הבאות:

public CharStack()	בונה מחסנית ריקה חדשה.
public void push(char value)	מכניסה את הערך הנתון למחסנית.
public char pop()	מוציאה את הערך שבראש המחסנית. אם המחסנית ריקה מחזירה '!'.
public boolean isEmpty()	מחזירה true אם המחסנית ריקה ו-false אחרת.
public int size()	מחזירה את מספר האיברים במחסנית.
Public String toString()	מחזירה ייצוג של המחסנית במחרוזת. המחרוזת מכילה את אברי המחסנית, כך שהשמאלי ביותר הוא זה שהוכנס ראשון למחסנית, ללא רווחים ביניהם.

בנוסף נתונה המחלקה IntQueue המממשת תור של טיפוס נתונים מסוג int (אין צורך לממש מחלקה זו). מחלקה זו הינה בעלת השיטות הבאות:

public IntQueue()	בונה תור ריק חדש.
public void enqueue(int value)	מכניסה את הערך הנתון לסוף התור.
public int dequeue()	מוציאה את הערך שבראש התור. אם התור ריק מחזירה -1.
public boolean isEmpty()	מחזירה true אם התור ריק ו-false אחרת.
Public int size()	מחזירה את מספר האיברים בתור.

```
public class Qa {

    public static void foo(int length){
        CharStack stack = new CharStack();
        fool(stack,length);
        System.out.println(stack);
        while(!stack.isEmpty()){
            if(stack.pop() == 'a'){
                stack.push('b');
                fool(stack, length);
                System.out.println(stack);
            }
        }
    }

    private static void fool(CharStack stack,int length){
        while(stack.size()<length){
            stack.push('a');
        }
    }
}
```

המשך השאלה בעמוד הבא

67.101
12/8/21

- (א) (7 נק') מהו הפלט של הפונקציה foo עבור הפרמטר 3?
- (ב) (13 נק') הסבירו במילים מה מבצעת הפונקציה foo (עליכם לתת תיאור ממצה של מה עושה השיטה באופן כללי ולא תיאור של מה עושה כל שורה בשיטה).
- (ג) (13 נק') הסבירו מה מבצעת השיטה foo3 ובאיזה אלגוריתם היא משתמשת על מנת לבצע זאת (עליכם לתת תיאור ממצה של מה עושה השיטה באופן כללי ולא תיאור של מה עושה כל שורה בשיטה):

```
public class Qb {  
    public static IntQueue foo3(IntQueue queue){  
        IntQueue queue1 = new IntQueue();  
        int size = queue.size();  
        for (int i = 0; i < size; i++){  
            int num1 = queue.dequeue();  
            for (int j = 0; j < queue.size(); j++){  
                int num2 = queue.dequeue();  
                if (num2 > num1){  
                    int tmp = num1;  
                    num1 = num2;  
                    num2 = tmp;  
                }  
                queue.enqueue(num2);  
            }  
            queue1.enqueue(num1);  
        }  
        return queue1;  
    }  
}
```

67. 101
ר"ר/כ

שאלה 4 (33 נקודות)

מיון יציב הוא מיון ששומר על הסדר המקורי בין פריטים שיש להם את אותו ערך בשדה לפיו ממיינים.

למשל, עבור פריטים שהם זוגות של `<character, _number>` ישנו המערך:

0	1	2	3	4
<code><a,1></code>	<code><b,3></code>	<code><c,2></code>	<code><m,2></code>	<code><e,2></code>

מיון יציב, שימיון את המערך לפי השדה `_number`, יהיה חייב לשמור על הסדר המקורי בין שלושת הפריטים שיש להם אותו ערך 2 בשדה `_number` ולכן ימיון כך:

0	1	2	3	4
<code><a,1></code>	<code><c,2></code>	<code><m,2></code>	<code><e,2></code>	<code><b,3></code>

לעומת זאת, מיון לא יציב לא מתחייב על שמירת הסדר הפנימי, ולכן אם ימיון לפי השדה `_number` ישנה אפשרות שייתן תוצאה כזאת:

0	1	2	3	4
<code><a,1></code>	<code><m,2></code>	<code><c,2></code>	<code><e,2></code>	<code><b,3></code>

הקוד הבא הוא גרסה של האלגוריתם Selection Sort (או Max Sort) עבור מערך של אובייקטים מטיפוס Pair שמתארים זוגות של `<character, _number>` כמו בדוגמה. הקוד ממיון את המערך לפי השדה `_number`:

```

1. public static void selectionSort(Pair[] arr) {
2.   for (int i = arr.length; i > 1; i--) {
3.     int maxIndex = 0;
4.     // Find max among [0...i-1]:
5.     for (int j = 0; j < i; j++) {
6.       if (arr[j]._number > arr[maxIndex]._number) {
7.         maxIndex = j;
8.       }
9.     }
10.    // Set the max to be last among [0...i-1]:
11.    slideInPlace(arr, maxIndex, i-1);
12.  }
13. }
14.
15. public static void slideInPlace(Object[] arr,
16.                                  int fromIndex, int toIndex) {
17.   Object tmp = arr[fromIndex];
18.   for (int i = fromIndex; i < toIndex; i++) {
19.     arr[i] = arr[i+1];
20.   }
21.   arr[toIndex] = tmp;
22. }

```

המשך השאלה בעמוד הבא

67-101
12/8/21

- (א) (12 נק') האם הקוד הנ"ל מממש מיון יציב? הסבירו. אם לדעתכם זהו מיון יציב, תנו דוגמה אחת שממחישה זאת. אם לדעתכם זהו אינו מיון יציב, תנו דוגמה שתוכיח זאת: דוגמה שמראה שהמיון משנה את הסדר הפנימי בין פריטים שיש להם אותו ערך בשדה לפיו ממיינים.
- (ב) (7 נק') איזו שורה תשנו בקוד, וכיצד, כדי להפוך את תכונת היציבות של המיון (אם המיון יציב, להפוך אותו ללא יציב, ואם הוא לא יציב להפוך אותו ליציב)? הסבירו.
- (ג) (9 נק') לצורך הסעיף הזה, לרשותכם פונקציה insertionSort שמממשת את אלגוריתם Insertion Sort באופן שיקיים מיון יציב, עבור מערך של אובייקטים כלליים, ולפי אובייקט השוואות שמממש ממשק Comparator בצורה המתאימה לטיפוס שבמערך. הפונקציה מסדרת את האובייקטים שבמערך מהקטן לגדול.

```
public interface Comparator {  
    // return negative integer if obj1 is smaller, zero  
    // if obj1 and obj2 are equal, or positive integer if  
    // obj1 is larger.  
    public int compare(Object obj1, Object obj2);  
}
```

```
public static void insertionSort(Object[]  
arr, Comparator comp) {  
    ...  
}
```

בנוסף, נתאר מחלקה בשם Person שבין השדות שלה ישנם שני שדות עבור שם פרטי ושם משפחה, ושתי מחלקות שמממשות את הממשק Comparator במיוחד עבור אובייקטים מטיפוס Person:

```
// Compares two objects of type Person according to  
// the first name, lexicographically  
public class PersonFirstNameComparator implements  
    Comparator {  
    public int compare(Object obj1, Object obj2){...}  
}  
  
// Compares two objects of type Person according to  
// the last name, lexicographically  
public class PersonLastNameComparator implements  
    Comparator {  
    public int compare(Object obj1, Object obj2) {...}  
}
```

עליכם לממש את הפונקציה sortPersonsByFullName שתמייין מערך של אובייקטים מטיפוס Person לפי שם: קודם לפי שם משפחה, ובתוך כל שם משפחה (עבור אנשים עם אותו שם משפחה) ממין לפי השם הפרטי. השתמש בפונקציה insertionSort שהוזכרה.

עליכם לממש את הפונקציה בצורה הכי קצרה (מבחינת שורות קוד).

```
public static void sortPersonsByFullName(Person[] arr)  
{...}
```

- (ד) (5 נק') מה סיבוכיות זמן הריצה של הפונקציה שמימשתם? הסבירו.

בהצלחה!